OS_Lab7_Experimental report

湖南大学信息科学与工程学院

计科 210X 甘晴void （学号 202108010XXX）

实验目的

- 理解操作系统的同步互斥的设计实现；
- 理解底层支撑技术：禁用中断、定时器、等待队列；
- 在ucore中理解信号量（semaphore）机制的具体实现；
- 理解管程机制，在ucore内核中增加基于管程（monitor）的条件变量（condition variable）的支持；
- 了解经典进程同步问题，并能使用同步机制解决进程同步问题。

实验内容

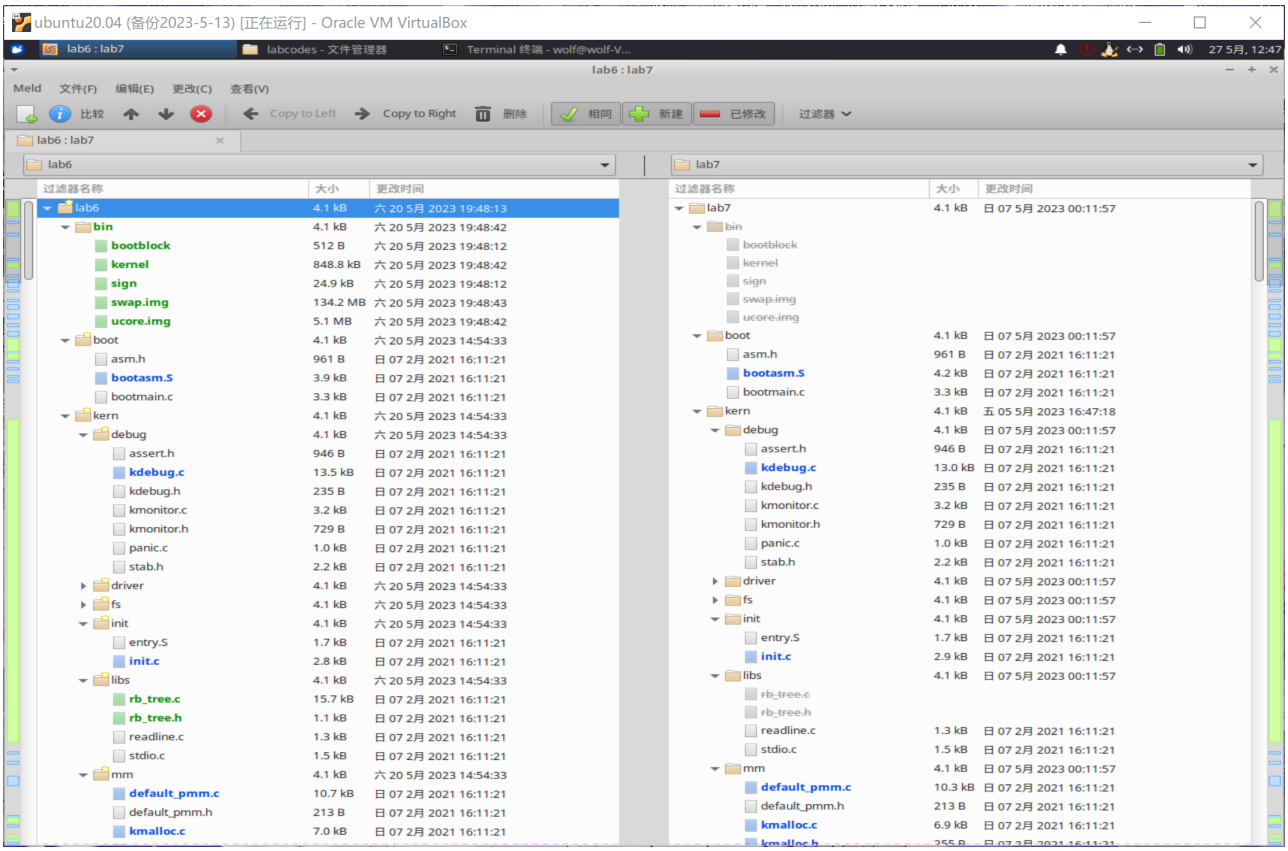
实验六完成了用户进程的调度框架和具体的调度算法，可调度运行多个进程。如果多个进程需要协同操作或访问共享资源，则存在如何同步和有序竞争的问题。本次实验，主要是熟悉ucore的进程同步机制—信号量（semaphore）机制，以及基于信号量的哲学家就餐问题解决方案。然后掌握管程的概念和原理，并参考信号量机制，实现基于管程的条件变量机制和基于条件变量来解决哲学家就餐问题。

在本次实验中，在kern/sync/check_sync.c中提供了一个基于信号量的哲学家就餐问题解法。同时还需完成练习，即实现基于管程（主要是灵活运用条件变量和互斥信号量）的哲学家就餐问题解法。

哲学家就餐问题描述如下：有五个哲学家，他们的生活方式是交替地进行思考和进餐。哲学家们公用一张圆桌，周围放有五把椅子，每人坐一把。在圆桌上有五个碗和五根筷子，当一个哲学家思考时，他不与其他人交谈，饥饿时便试图取用其左、右最靠近他的筷子，但他可能一根都拿不到。只有在他拿到两根筷子时，方能进餐，进餐完后，放下筷子又继续思考。

练习**0**：填写已有实验

`lab7` 会依赖 `lab1~lab6`，我们需要把做的 `lab1~lab6` 的代码填到 `lab7` 中缺失的位置上面。练习 0 就是一个工具的利用。这里我使用的是 `Linux` 下的系统已预装好的 `Meld Diff Viewer` 工具。和 `lab6` 操作流程一样，我们只需要将已经完成的 `lab1~lab6` 与待完成的 `lab7` (由于 `lab7` 是基于 `lab1~lab6` 基础上完成的，所以这里只需要导入 `lab6`)分别导入进来，然后对比进行删改即可。



其他需要修改的地方主要有以下六个文件，通过对比复制完成即可：

- proc.c
- default_pmm.c
- pmm.c
- swap_fifo.c
- vmm.c
- trap.c
- sche.c

根据试验要求，我们需要对部分代码进行改进，这里讲需要改进的地方只有一处：

trap.c() 函数

修改的部分如下：

```
static void trap_dispatch(struct trapframe *tf) {
    ++ticks;
    /* 注销掉下面这一句 因为这一句被包含在了 run_timer_list()
        run_timer_list() 在之前的基础上 加入了对 timer 的支持 */
    // sched_class_proc_tick(current);
    run_timer_list();
}
```

练习1:理解内核级信号量的实现和基于内核级信号量的哲学家就餐问题（不需要编码）

> 完成练习0后，建议大家比较一下（可用*meld*等文件*diff*比较软件）个人完成的*lab6*和练习0完成后的刚修改的*lab7*之间的区别，分析了解*lab7*采用信号量的执行过程。执行`make grade`，大部分测试用例应该通过。
>
> 请在实验报告中给出内核级信号量的设计描述，并说明其大致执行流程。
>
> 请在实验报告中给出给用户态进程/线程提供信号量机制的设计方案，并比较说明给内核级提供信号量机制的异同。

比较

分析lab6和lab7的区别可知，lab7主要多了sync文件夹，也就是sem.c , monitor.c , wait.c 等文件，增添了信号量等操作从而使得操作系统支持进程同步。在trap.c中，当发生时钟中断时，需要调用run_time_list函数。在lab6中，互斥通过lock实现，在lab7中则使用信号量实现互斥，此外增加了等待队列、计时器、禁用中断等。之后将具体分析。

哲学家就餐问题

具体问题在上面已经描述过了。

解决该问题的目标是保证没有死锁和饥饿的情况发生，且尽可能让更多哲学家吃到东西。本次实验需要分别使用信号量和条件变量解决这个问题，信号量的实现已经完成了，练习1是理解信号量解决方法。

```
//check_sync.c

#define N 5 /* 哲学家数目 */
void check_sync(void){
    int i;
    //check semaphore
```

```
    sem_init(&mutex, 1);//初始化mutex信号量为1，mutex信号量也就是缓冲区的
锁
    for(i=0;i<N;i++){
        sem_init(&s[i], 0); //初始化5个信号量为0
        int pid = kernel_thread(philosopher_using_semaphore, (void
*)i, 0);
        //调用kernel_thread创建了5个内核线程，对应5位哲学家
        if (pid <= 0) {
            panic("create No.%d philosopher_using_semaphore
failed.\n");
            //子线程创建失败错误处理机制
        }
        philosopher_proc_sema[i] = find_proc(pid);
        set_proc_name(philosopher_proc_sema[i],
"philosopher_sema_proc");
    }

//每个哲学家都按照下面的函数所示过程尝试吃饭
#define TIMES  4 /* 吃多少次次饭，这里我们设定为4 */
int philosopher_using_semaphore(void * arg) /* i：哲学家号码，从0到N-1
*/
{
    int i, iter=0;
    i=(int)arg;//i个哲学家
    cprintf("I am No.%d philosopher_sema\n",i);
    while(iter++<TIMES)
    //先进行思考，然后尝试拿起两只叉子，拿到叉子后就餐，最后再将叉子放回桌子，这样
做TIMES次
    { /* 无限循环 */
        cprintf("Iter %d, No.%d philosopher_sema is
thinking\n",iter,i); /* 哲学家正在思考 */
        do_sleep(SLEEP_TIME);
        phi_take_forks_sema(i); /* 需要两只叉子，或者阻塞 */
        cprintf("Iter %d, No.%d philosopher_sema is
eating\n",iter,i); /* 进餐 */
        do_sleep(SLEEP_TIME);
        phi_put_forks_sema(i); /* 把两把叉子同时放回桌子 */
    }
    cprintf("No.%d philosopher_sema quit\n",i);
    return 0;
}
```

```
//其中就餐和思考的过程是通过do_sleep进行睡眠模拟的。
//do_sleep 会将当前进程的 state 设置为 SLEEPING，然后为进程创建一个定时器，并添加到定时器列表中，最后调用 schedule 让出 CPU 给其他进程。
int do_sleep(unsigned int time) {
    if (time == 0) {
        return 0;
    }
    bool intr_flag;
    local_intr_save(intr_flag);
    timer_t __timer, *timer = timer_init(&__timer, current, time);
    current->state = PROC_SLEEPING;//将当前进程的 state 设置为 SLEEPING
    current->wait_state = WT_TIMER;
    add_timer(timer);            //添加定时器
    local_intr_restore(intr_flag);
    schedule();                  //调度其他进程运行
    del_timer(timer);
    return 0;
}

//获取两个叉子
void phi_take_forks_sema(int i) /* i：哲学家号码从0到N-1 */{
        down(&mutex);                /* 进入临界区 */
        state_sema[i]=HUNGRY;        /* 记录下哲学家i饥饿的事实 */
        phi_test_sema(i);            /* 试图得到两只叉子 */
        up(&mutex);                  /* 离开临界区 */
        down(&s[i]);                 /* 如果得不到叉子就阻塞 */
    //若之前调用phi_test_sema(i)时，该哲学家拿到了两个叉子，s[i]就是1，这里只会减为0，而只有小于0才会进入等待，该哲学家是可以继续的；但是如果该哲学家之前没有获取两个叉子，这里就被阻塞了。
    //这里书上的表述跟实验表述有差异，书上是先减去value值再判定的，实验是先判定再减去value值的，但是本质都是一样的，只会相差一个等号的判定。
}

//尝试看是否能够真的获取这两个叉子
void phi_test_sema(i) /* i：哲学家号码从0到N-1 */{
    if(state_sema[i]==HUNGRY && state_sema[LEFT]!=EATING && state_sema[RIGHT]!=EATING){
        //左右两个哲学家都不在使用叉子，故本哲学家可以拿这两个叉子
        state_sema[i]=EATING;        //将i状态修改，i拿起了叉子并可以就餐
        up(&s[i]);                   //信号量+1
    }
```

```
}

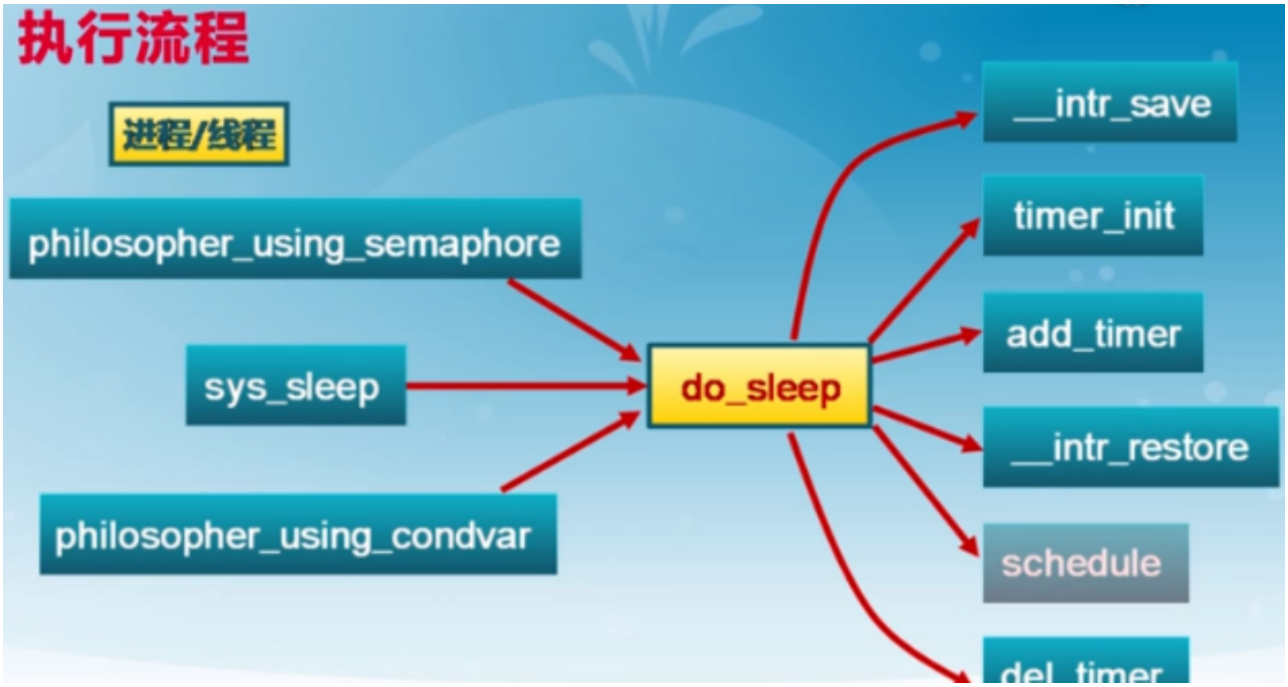
//放下两个叉子
#define LEFT (i-1+N)%N
#define RIGHT (i+1)%N
void phi_put_forks_sema(int i) /* i：哲学家号码从0到N-1 */{
        down(&mutex);                   /* 进入临界区 */
        state_sema[i]=THINKING;         /* 哲学家进餐结束 */
        phi_test_sema(LEFT);            /* 看一下左邻居现在是否能进餐 */
        phi_test_sema(RIGHT);           /* 看一下右邻居现在是否能进餐 */
        up(&mutex);                     /* 离开临界区 */
}

//注意：
//对哲学家的状态进行修改，而哲学家状态的列表是全局数据，因此需要使用mutex保护临界区
```

sleep函数图解如下：



全流程如下：



内核级信号量的设计描述与大致的执行流程

信号量的数据结构定义如下

```
typedef struct {
    int value;                          //信号量的当前值
    wait_queue_t wait_queue;            //信号量对应的等待队列
} semaphore_t;
```

在ucore中最重要的信号量操作是P操作函数down(semaphore_t *sem)和V操作函数up(semaphore_t *sem)

P是荷兰语的Passeren，相当于英文的pass, V是荷兰语的Verhoog,相当于英文中的increment。

value的含义如下：

- value>0， 表示共享资源的空闲数
- vlaue<0， 表示该信号量的等待队列里的进程数
- value=0， 表示等待队列为空

这两个函数的具体实现如下：

down函数

value>0表示可以获取信号量，修改信号量并返回

value<=表示无法获取信号量，将当前的进程加入到等待队列，运行调度器选择另外一个进程执行。如果被V操作唤醒，则把自身关联的wait从等待队列中删除（此过程需要先关闭中断，完成后开启中断）

```
static __noinline uint32_t __down(semaphore_t *sem, uint32_t
wait_state) {
    bool intr_flag;
    local_intr_save(intr_flag);                                    //关闭中
断
    if (sem->value > 0) {//value>0，表示可以获取信号量
        sem->value --;
        local_intr_restore(intr_flag);                             //开启中
断
        return 0;                                                  //>0直接
返回，表示许可继续
    }
    //value<=0，无法获得信号量
    wait_t __wait, *wait = &__wait;
    wait_current_set(&(sem->wait_queue), wait, wait_state); //将当前
进程加入等待队列，设置睡眠状态
    local_intr_restore(intr_flag);                                 //打开中
断并调度其他进程
    schedule();
    /* 被唤醒 */
    local_intr_save(intr_flag);
    wait_current_del(&(sem->wait_queue), wait);                    //从等待
队列删除当前进程
    local_intr_restore(intr_flag);

    if (wait->wakeup_flags != wait_state) {
        return wait->wakeup_flags;                                 //不是信
号量导致的休眠则返回休眠原因
    }
    return 0;
}
```

屏蔽中断与使能中断



up函数

如果信号量对应的wait queue中没有进程在等待，直接把信号量的value加一，然后打开中断返回。

如果有进程在等待且进程等待的原因是semophore设置的，则调用wakeup_wait函数将waitqueue中等待的第一个wait删除，且把此wait关联的进程唤醒，最后开中断返回

```
static __noinline void __up(semaphore_t *sem, uint32_t wait_state)
{
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        wait_t *wait;
        // 如果当前等待队列中没有线程等待，则value照常+1
        if ((wait = wait_queue_first(&(sem->wait_queue))) == NULL)
{
            sem->value ++;
        }
        // 否则如果当前等待队列中存在线程正在等待，则唤醒该线程并开始执行对应代码
        else {
            assert(wait->proc->wait_state == wait_state);
            wakeup_wait(&(sem->wait_queue), wait, wait_state, 1);
        }
    }
    local_intr_restore(intr_flag);
}
```

这两个函数实现了信号量上进程的唤醒和睡眠操作，唤醒是将进程设置为可运行状态并加入运行队列，而睡眠则是将当前进程设置为睡眠状态，这样该进程将不会被调度，直到被唤醒。

用户态进程/线程提供信号量机制的设计方案

至少应该有如下的操作：

- sem_init：生成一个信号量，并初始化值。
- sem_post：V操作，可以直接调用up函数完成
- sem_wait：P操作，可以直接调用down函数完成

一种综合考虑过之后的实现为：

```
/*Initialize semaphore object SEM to VALUE.  If PSHARED then share
it
   with other processes.  */
int sem_init (sem_t *__sem, int __pshared, unsigned int __value);
/* Free resources associated with semaphore object SEM.  */
// 将信号量所使用的资源全部释放
int sem_destroy (sem_t *__sem);

/* Open a named semaphore NAME with open flags OFLAG.  */
// 开启一个新信号量，并使用给定的flag来指定其标志
sem_t *sem_open (const char *__name, int __oflag, ...);

/* Close descriptor for named semaphore SEM.  */
// 将当前信号量所使用的描述符关闭
int sem_close (sem_t *__sem);

/* Remove named semaphore NAME.  */
int sem_unlink (const char *__name);

/* Wait for SEM being posted.

   This function is a cancellation point and therefore not marked
with
   __THROW.  */
// 一个P操作，如果sem value > 0，则sem value--；否则阻塞直到sem value > 0
int sem_wait (sem_t *__sem);

/* Test whether SEM is posted.  */
int sem_trywait (sem_t *__sem);

/* Post SEM.  */
// 一个V操作，把指定的信号量 sem 的值加 1，唤醒正在等待该信号量的任意线程。
```

```
int sem_post (sem_t *__sem);

/* Get current value of SEM and store it in *SVAL.  */
// 获取当前信号量的值
int sem_getvalue (sem_t *__restrict __sem, int *__restrict __sval);
```

给内核级提供信号量机制的异同

相同点：

- 其核心的实现逻辑是一样的

不同点：

- 内核态的信号量机制可以直接调用内核的服务，而用户态的则需要通过内核提供的接口来访问内核态服务，这其中涉及到了用户态转内核态的相关机制。
- 内核态的信号量存储于内核栈中；但用户态的信号量存储于用户栈中。

练习2: 完成内核级条件变量和基于内核级条件变量的哲学家就餐问题（需要编码）

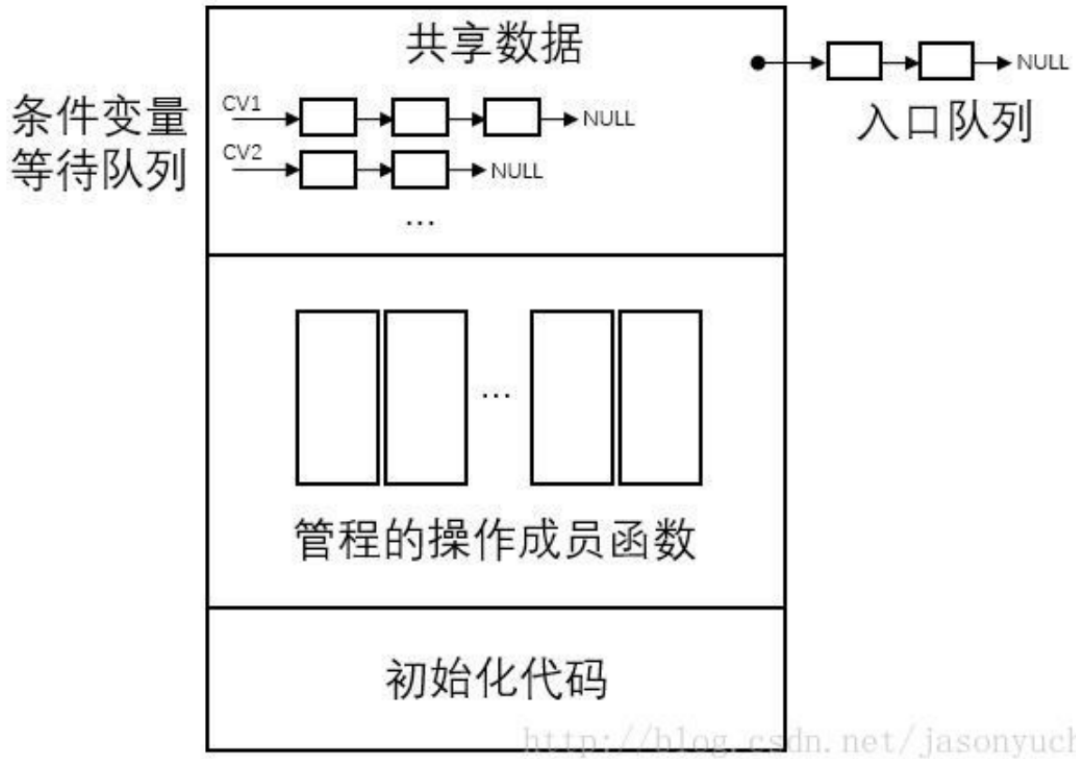
> 首先掌握管程机制，然后基于信号量实现完成条件变量实现，然后用管程机制实现哲学家就餐问题的解决方案（基于条件变量）。
>
> 执行：`make grade`。如果所显示的应用程序检测都输出*ok*，则基本正确。如果只是某程序过不去，比如*matrix.c*，则可执行
>
> ```
> make run-matrix
> ```
>
> 命令来单独调试它。大致执行结果可看附录。
>
> 请在实验报告中给出内核级条件变量的设计描述，并说明其大致执行流程。
>
> 请在实验报告中给出给用户态进程/线程提供条件变量机制的设计方案，并比较说明给内核级提供条件变量机制的异同。
>
> 请在实验报告中回答：能否不用基于信号量机制来完成条件变量？如果不能，请给出理由，如果能，请给出设计说明和具体实现。

1、管程机制

Hansan为管程所下的定义：“一个管程定义了一个数据结构和能为并发进程所执行（在该数据结构上）的一组操作，这组操作能同步进程和改变管程中的数据”。

因此管程由四部分组成：

- 管程内部的共享变量
- 管程内部的条件变量
- 管程内部并发执行的进程
- 对局部于管程内部的共享数据设置初始值的语句



管程的作用相当于将共享变量、涉及该共享变量的进程、互斥量及操作全部封装起来进行管理。所有进程要访问临界资源时，都必须经过管程才能进入，而管程每次只允许一个进程进入管程，从而需要确保进程之间互斥。

很多情况下，进程需要等待某个条件满足后才会继续运行。为了实现这种等待，管程需要提供条件变量。最终一个管程的结构定义如下：

```
// 管程的数据结构
typedef struct monitor{
    semaphore_t mutex; // 二值信号量 用来互斥访问管程
    semaphore_t next; // 用于条件同步 用于发出signal操作的进程等条件为真之前进入睡眠
    int next_count; // 记录睡在 signal 操作的进程数
    condvar_t *cv; // 条件变量
} monitor_t;
```

管程中的条件变量cv通过执行cond_wait，会使得等待某个条件C为真的进程能够离开管程并睡眠，且让其他进程进入管程继续执行；而进入管程的某进程设置条件C为真并执行cond_signal时，能够让等待某个条件C为真的睡眠进程被唤醒，从而继续进入管程中执行。

信号量next和整形变量next_count用于配合cv操作。由于管程只有一个进程可以进入，当前进程cond_signal唤醒了等待状态睡眠的进程后，将睡眠在管程的next上，当有进程再次等待条件时，再将该进程唤醒。这个同步过程是通过信号量next完成的；而next_count表示了由于发出cond_signal而睡眠的进程个数。

举例

只考虑2个进程进入管程的情况，大致的过程如下：

进程1获取管程的锁

->条件不满足，进程1调用cond_wait释放锁在条件变量上等待(进程1在管程内)

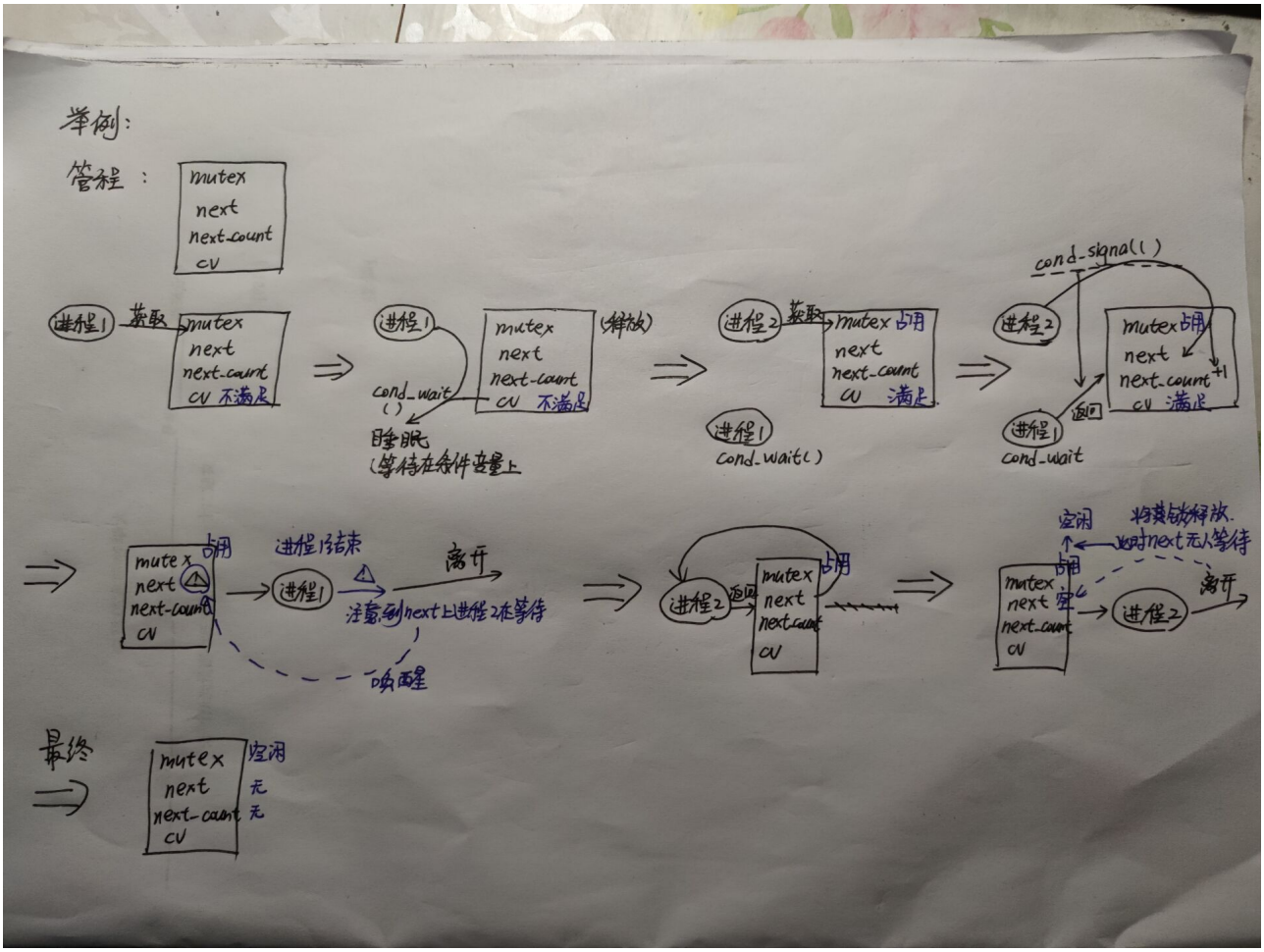
->进程2获取管程的锁

->条件达成，进程2cond_signal唤醒进程1，进程2等待在管程的next上(进程2在管程内)

->进程1从cond_wait返回，完成工作后离开管程，发现next有进程等待，唤醒进程2

->进程2从cond_signal返回，完成工作后离开管程，next没有进程等待，释放管程锁

整个过程中，只有一个进程在管程内，对共享变量进行访问，并通过条件变量实现了条件等待。其中比较重要的部分是：发出cond_signal的进程，需要等待另一个进程唤醒后离开管程，因此等待在管程的信号量next上。这种等待与条件等待是不同的。进入管程时，是由mutex保证只有一个进程可以进入管程，但是如果一个进程在管程内cond_wait等待，释放了锁，就可能会有另一个进程进入管程，两个进程不能同时运行，必须有一个在等待，因此管程中才提供了信号量next，保证管程内只有一个进程运行。



2、条件变量

```
// 条件变量数据结构
typedef struct condvar{
    semaphore_t sem; // 用于条件同步 用于发出wait操作的进程等待条件为真之前进入睡眠
    int count; // 记录睡在 wait 操作的进程数(等待条件变量成真)
    monitor_t * owner; // 所属管程
} condvar_t;
```

通过条件变量中的信号量，可以使等待某个特定条件的进程睡眠，而另一个完成该特定条件的进程可以将其唤醒。

cond_wait函数

其中等待条件变量的操作通过调用cond_wait完成。在cond_wait中，首先要查看next上是否有进程在睡眠，这些进程因发出cond_signal而睡眠，当前进程需要唤醒其中一个进程。如果没有则唤醒一个唤醒因为互斥条件mutex无法进入管程的进程。然后自己等待在条件变量上，直到条件满足，被cond_signal唤醒。

```
void
cond_wait (condvar_t *cvp) {
    cprintf("cond_wait begin:  cvp %x, cvp->count %d, cvp->owner->next_count %d\n", cvp, cvp->count, cvp->owner->next_count);
    cvp->count++;                                          //等待在条件变量上的进程数+1
    if(cvp->owner->next_count > 0)                         //如果有则唤醒next上的进程，锁传递
        up(&(cvp->owner->next));
    else
        up(&(cvp->owner->mutex));                          //没有则释放锁
    down(&(cvp->sem));
    cvp->count --;
    cprintf("cond_wait end:  cvp %x, cvp->count %d, cvp->owner->next_count %d\n", cvp, cvp->count, cvp->owner->next_count);
}
```

cond_signal函数

cond_signal负责唤醒进程。调用up唤醒一个等待在条件变量上的进程，自己则调用down进行睡眠，等待唤醒。

```
void
cond_signal (condvar_t *cvp) {
   cprintf("cond_signal begin: cvp %x, cvp->count %d, cvp->owner->next_count %d\n", cvp, cvp->count, cvp->owner->next_count);
   if(cvp->count>0){                                       //判断是否有进程等待
       cvp->owner->next_count ++;
//next_count++，自己将要睡眠
       up(&(cvp->sem));                                    //唤醒一个在条件变量等待的进程
       down(&(cvp->owner->next));                          //自己睡眠，等待同步
       cvp->owner->next_count--;
   }
   cprintf("cond_signal end: cvp %x, cvp->count %d, cvp->owner->next_count %d\n", cvp, cvp->count, cvp->owner->next_count);
}
```

3、条件变量实现的哲学家就餐问题

check_sync的后半部分为测试条件变量实现的哲学家就餐问题。与测试信号量实现的哲学家就餐问题相同，创建五个线程，调用对应的主函数。不同的地方在于与信号量实现中需要每个哲学家定义一个信号量相比，只需要创建并初始化一个管程，在管程中将保存需要用到的五个条件变量。

```
//修改check_sync

//check condition variable
    monitor_init(&mt, N);                                    //使用管
程mt完成哲学家就餐问题
    for(i=0;i<N;i++){
        state_condvar[i]=THINKING;
        int pid = kernel_thread(philosopher_using_condvar, (void
*)i, 0);
        if (pid <= 0) {
            panic("create No.%d philosopher_using_condvar
failed.\n");
        }
        philosopher_proc_condvar[i] = find_proc(pid);
        set_proc_name(philosopher_proc_condvar[i],
"philosopher_condvar_proc");
    }
}

//就餐过程与信号量实现的哲学家就餐问题基本相同，只有获取餐叉和放下餐叉使用的是条件
变量实现的函数。
//这一部分不需要太多做改动
int philosopher_using_condvar(void * arg) { /* arg is the No. of
philosopher 0~N-1*/

    int i, iter=0;
    i=(int)arg;
    cprintf("I am No.%d philosopher_condvar\n",i);
    while(iter++<TIMES)
    { /* iterate*/
        cprintf("Iter %d, No.%d philosopher_condvar is
thinking\n",iter,i); /* thinking */
        do_sleep(SLEEP_TIME);
        phi_take_forks_condvar(i);
//使用条件变量的拿起叉子
```

```
        /* need two forks, maybe blocked */
        cprintf("Iter %d, No.%d philosopher_condvar is
eating\n",iter,i);   /* eating */
        do_sleep(SLEEP_TIME);
        phi_put_forks_condvar(i);
//使用条件变量的放下叉子
        /* return two forks back*/
    }
    cprintf("No.%d philosopher_condvar quit\n",i);
    return 0;
}

//获取叉子，获取不到则将等待（这是练习需要完成的内容）
void phi_take_forks_condvar(int i) {
     down(&(mtp->mutex));//进入管程，获取锁
//--------into routine in monitor--------------
     state_condvar[i] = HUNGRY;
     phi_test_condvar(i);
     if(state_condvar != EATING)    //不使用while也没有问题，因为状态只有
自己能改回thinking
        cond_wait(&mtp->cv[i]);
    //在持有锁的情况下调用phi_test_condvar尝试获取叉子，如果获取到了叉子，状态
将被修改为就餐状态，否则状态不变，则需要通过cond_wait睡眠在对应的条件变量上。
//--------leave routine in monitor--------------
      if(mtp->next_count>0)
         up(&(mtp->next));
      else
         up(&(mtp->mutex));          //释放锁
}

//尝试获取叉子，根据状态判断是否可以获取
void phi_test_condvar (i) {
    if(state_condvar[i]==HUNGRY&&state_condvar[LEFT]!=EATING
            &&state_condvar[RIGHT]!=EATING) {
        cprintf("phi_test_condvar: state_condvar[%d] will
eating\n",i);
        state_condvar[i] = EATING ;
        cprintf("phi_test_condvar: signal self_cv[%d] \n",i);
        cond_signal(&mtp->cv[i]) ;
    }
}
```

```
//放下叉子（这是练习需要完成的内容）
//将当前哲学家设置为THINKING状态，然后测试左右的哲学家是否可以获取叉子，如果可以获取，则将其使用cond_signal唤醒。
void phi_put_forks_condvar(int i) {
     down(&(mtp->mutex));
//--------into routine in monitor--------------
     state_condvar[i] = THINKING;
     phi_test_condvar(LEFT);
     phi_test_condvar(RIGHT);
//--------leave routine in monitor--------------
     if(mtp->next_count>0)
        up(&(mtp->next));
     else
        up(&(mtp->mutex));
}
```

4、用户态进程/线程条件变量机制

与用户态下的信号量类似，用户态下的条件变量同样不可以在内核态下直接使用管程和条件变量实现。因为管程和条件变量也是依赖信号量，通过up和down函数中开关中断实现操作的原子性，在用户态下是不能使用的。因此，用户态下的条件变量机制也需要使用系统调用来完成。与信号量类似，至少需要三个系统调用接口：

- cond_init：创建并初始化条件变量
- cond_wait：使进程等待在条件变量上
- cond_signal：唤醒等待在条件变量的进程

同样，条件变量的位置需要解决，如果存放在内核中，用户态使用内核地址的条件变量指针可能存在问题。此问题待解决。

5、不基于信号量的条件变量机制

使用信号量来实现条件变量机制，实际上是使用了信号量的等待队列，并且使用了值来记录对资源的访问情况。由于与管程结合，直接使用信号量更加方便的实现了管程，使条件变量和锁配合使用。

对于一个简单的，不与其他机制结合的条件变量，不需要复杂的实现，也不需要值来记录，只需要一个等待队列。即按照OSTEP书中的定义：条件变量是一个显示队列，当某些条件不满足时，线程或进程可以将自己加入队列，等待该条件；另外某个线程或进程，改变了上述条件时，就可以唤醒一个或多个等待线程。只需要一个等待队列，就可以实现一个简单的

条件变量。而对条件变量的wait和signal操作，可以和信号量一样，使用开关中断的方式保证原子性。

实现简单的条件变量，首先进行定义，条件变量只需要作为一个显式队列，而与条件变量相关的操作有四个，cond_init对条件变量的队列进行初始化，cond_wait使当前进程等待，cond_signal唤醒一个等待进程，cond_broadcast唤醒所有等待进程。条件变量通常是与锁配合使用的，在进入临界区时会获取锁，如果调用cond_wait进行等待，则需要释放锁，因此cond_wait应该传入一个锁。但是由于实验中是使用开关中断实现了锁的功能，所以这里不传入锁，而是在传入标志中断开关的flag，从而打开中断。

```
//定义一个只有等待队列的条件变量cond_t
typedef struct {
    wait_queue_t wait_queue;
} cond_t;
//相关操作
void cond_init(cond_t *cond );
void cond_wait(cond_t *cond,  bool intr_flag);
void cond_signal(cond *cond);
bool cond_broadcast(cond *cond);
```

接下来对相关函数进行实现，cond_init只需要初始化等待队列，等待和唤醒，仿照信号量的实现，定义**wait**和signal完成。并且向这些函数额外传入一个等待状态。等待状态是在proc.h中定义的，可以仿照定义一个WT_KCOND表示因内核条件变量等待。

```
#define WT_KCOND 0x00001000          //仿照WT_KSEM在proc.h中定义
//条件变量初始化
void
cond_init(cond_t *cond) {
    wait_queue_init(&(cond->wait_queue));
}
//等待与唤醒
void
cond_wait(cond_t *cond) {
    __wait(sem, WT_KCOND);
}
void
cond_signal(cond_t *cond) {
    __signal(sem, WT_KCOND, 0);      //0表示不广播唤醒所有进程
}
void
cond_broadcast(cond_t *cond) {
    __signal(sem, WT_KCOND, 1);      //广播唤醒所有进程
```

```
}
```

__wait的实现只需要将当前进程加入等待队列，加入队列之前需要释放锁。

```
static __noinline uint32_t __wait(cond_t *cond, bool intr_flag,
uint32_t wait_state) {
    //现在是关闭中断的状态
    wait_t __wait, *wait = &__wait;
    wait_current_set(&(cond->wait_queue), wait, wait_state);    //
让当前进程等待
    local_intr_restore(intr_flag);                              //
开中断
    schedule();                                                 //
调度其他进程运行
    local_intr_save(intr_flag);                                 //
重新打开中断
    wait_current_del(&(sem->wait_queue), wait);
    if (wait->wakeup_flags != wait_state) {
        return wait->wakeup_flags;
    }
    return 0;                                                   //
以关中断状态返回
}
```

`__signal`和`__broadcast`将等待在条件变量的等待队列上的进程唤醒，区别在于唤醒进程的数量。

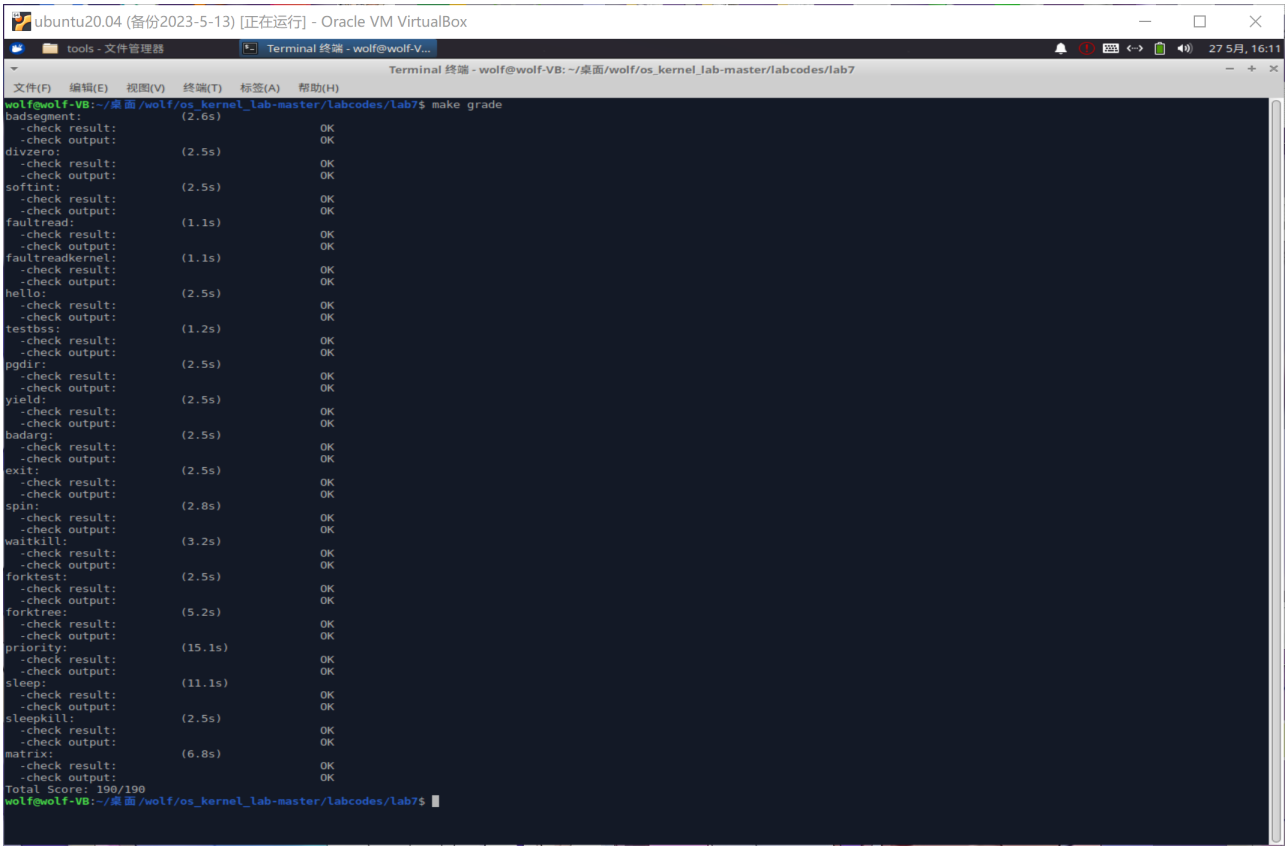
```
static __noinline void __signal(cond_t *cond, uint32_t wait_state,
bool broadcast) {
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        wait_t *wait;
        while ((wait = wait_queue_first(&(cond->wait_queue))) !=
NULL) {    //不为空
            assert(wait->proc->wait_state == wait_state);
            wakeup_wait(&(cond->wait_queue), wait, wait_state, 1);
            if(!broadcast) break;
//只唤醒一个
        }
    }
    local_intr_restore(intr_flag);
}
```

这样就实现了一个最简单的条件变量，配合开关中断就可以使用了。

```
//使用条件变量，假设有一个等待条件done
//等待条件的进程：
    bool intr_flag;
    local_intr_save(intr_flag);
//相当于上锁
    {
        while(done == 0){
            cond_wait(&cond, intr_flag);
        }
        等待条件成立后执行的代码...
    }
    local_intr_restore(intr_flag)
//改变等待条件的进程
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        done = 1;
        cond_signal(&cond);
    }
    local_intr_restore(intr_flag)
```

以上实现的条件变量只是根据书中给出的定义所完成的基本实现，仅仅作为等待队列使用，真正的实现可能要更加复杂，例如ucore中的管程和条件变量结合的实现。因此这个条件变量实现只是辅助对条件变量使用和概念的理解，不能使用。但是显然的是，只要有等待队列就可以实现条件变量，不一定需要信号量才可以实现条件变量。

实验结果

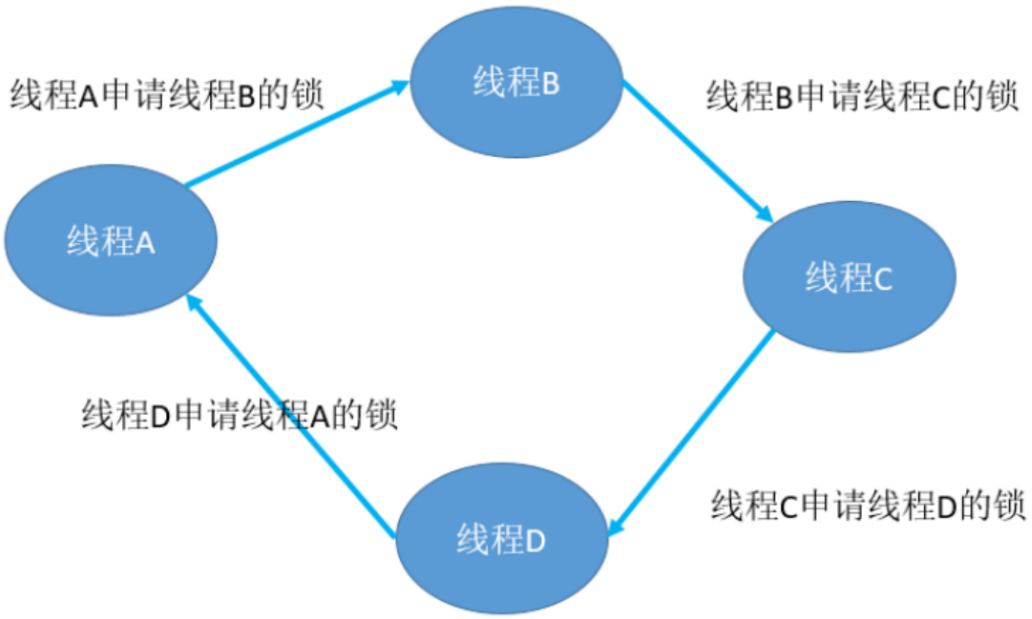


Challenge 1 ： 在ucore中实现简化的死锁和重入探测机制

在ucore下实现一种探测机制，能够在多进程/线程运行同步互斥问题时，动态判断当前系统是否出现了死锁产生的必要条件，是否产生了多个进程进入临界区的情况。 如果发现，让系统进入monitor状态，打印出你的探测信息。

死锁发生的四个必要条件

- 互斥条件：一个资源同一时间能且只能被一个线程访问；
- 不可掠夺：当资源被一个线程占用时，其他线程不可抢夺该资源；
- 请求与等待：当资源被一个线程占用时，其他线程只能等待资源的释放再拥有；
- 循环等待：指的是若干线程形成头尾相接的情况，将所有资源都占用导致的整体死锁或局部死锁。



这里试图通过锁进程间资源的依赖关系，使用数据结构有向图来构建。如线程A想要获取线程B已占有的资源，则建立一条A指向B的关系。

但可惜的是，只是实现了对于死锁的监测，并没有将它与ucore系统结合起来。

```
#define _GNU_SOURCE
#include <dlfcn.h>

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#include <stdlib.h>
#include <stdint.h>

#define THREAD_NUM      10

typedef unsigned long int uint64;

typedef int (*pthread_mutex_lock_t)(pthread_mutex_t *mutex);
pthread_mutex_lock_t pthread_mutex_lock_f;

typedef int (*pthread_mutex_unlock_t)(pthread_mutex_t *mutex);
pthread_mutex_unlock_t pthread_mutex_unlock_f;


#if 1 // graph
```

```
#define MAX     100

enum Type {PROCESS, RESOURCE};

struct source_type {
    uint64 id;
    enum Type type;
    uint64 lock_id;
    int degress;
};

struct vertex {
    struct source_type s;
    struct vertex *next;
};

struct task_graph {
    struct vertex list[MAX];
    int num;
    struct source_type locklist[MAX];
    int lockidx;
    pthread_mutex_t mutex;
};

struct task_graph *tg = NULL;
int path[MAX+1];
int visited[MAX];
int k = 0;
int deadlock = 0;

struct vertex *create_vertex(struct source_type type) {
    struct vertex *tex = (struct vertex *)malloc(sizeof(struct
vertex ));
    tex->s = type;
    tex->next = NULL;
    return tex;
}

int search_vertex(struct source_type type) {
    int i = 0;
    for (i = 0;i < tg->num;i ++) {
```

```
        if (tg->list[i].s.type == type.type && tg->list[i].s.id ==
type.id) {
            return i;
        }
    }
    return -1;
}

void add_vertex(struct source_type type) {
    if (search_vertex(type) == -1) {
        tg->list[tg->num].s = type;
        tg->list[tg->num].next = NULL;
        tg->num ++;
    }
}


int add_edge(struct source_type from, struct source_type to) {

    add_vertex(from);
    add_vertex(to);
    struct vertex *v = &(tg->list[search_vertex(from)]);
    while (v->next != NULL) {
        v = v->next;
    }
    v->next = create_vertex(to);
}


int verify_edge(struct source_type i, struct source_type j) {
    if (tg->num == 0) return 0;

    int idx = search_vertex(i);
    if (idx == -1) {
        return 0;
    }
    struct vertex *v = &(tg->list[idx]);
    while (v != NULL) {
        if (v->s.id == j.id) return 1;
        v = v->next;
    }
    return 0;
```

```
}

int remove_edge(struct source_type from, struct source_type to) {
    int idxi = search_vertex(from);
    int idxj = search_vertex(to);

    if (idxi != -1 && idxj != -1) {
        struct vertex *v = &tg->list[idxi];
        struct vertex *remove;
        while (v->next != NULL) {
            if (v->next->s.id == to.id) {
                remove = v->next;
                v->next = v->next->next;
                free(remove);
                break;
            }
            v = v->next;
        }
    }
}


void print_deadlock(void) {
    int i = 0;
    printf("deadlock : ");
    for (i = 0;i < k-1;i ++) {
        printf("%ld --> ", tg->list[path[i]].s.id);
    }
    printf("%ld\n", tg->list[path[i]].s.id);
}

int DFS(int idx) {

    struct vertex *ver = &tg->list[idx];
    if (visited[idx] == 1) {
        path[k++] = idx;
        print_deadlock();
        deadlock = 1;
        return 0;
    }

    visited[idx] = 1;
```

```
    path[k++] = idx;

    while (ver->next != NULL) {
        DFS(search_vertex(ver->next->s));
        k --;
        ver = ver->next;
    }
    return 1;
}


int search_for_cycle(int idx) {
    struct vertex *ver = &tg->list[idx];
    visited[idx] = 1;
    k = 0;
    path[k++] = idx;

    while (ver->next != NULL) {
        int i = 0;
        for (i = 0;i < tg->num;i ++) {
            if (i == idx) continue;
            visited[i] = 0;
        }
        for (i = 1;i <= MAX;i ++) {
            path[i] = -1;
        }
        k = 1;
        DFS(search_vertex(ver->next->s));
        ver = ver->next;
    }
}


#if 0
int main() {
    tg = (struct task_graph*)malloc(sizeof(struct task_graph));
    tg->num = 0;

    struct source_type v1;
    v1.id = 1;
    v1.type = PROCESS;
    add_vertex(v1);
```

```
    struct source_type v2;
    v2.id = 2;
    v2.type = PROCESS;
    add_vertex(v2);

    struct source_type v3;
    v3.id = 3;
    v3.type = PROCESS;
    add_vertex(v3);

    struct source_type v4;
    v4.id = 4;
    v4.type = PROCESS;
    add_vertex(v4);


    struct source_type v5;
    v5.id = 5;
    v5.type = PROCESS;
    add_vertex(v5);


    add_edge(v1, v2);
    add_edge(v2, v3);
    add_edge(v3, v4);
    add_edge(v4, v5);
    add_edge(v3, v1);

    search_for_cycle(search_vertex(v1));

}
#endif

#endif

void check_dead_lock(void) {

    int i = 0;
    deadlock = 0;
    for (i = 0;i < tg->num;i ++) {
        if (deadlock == 1) break;
```

```
        search_for_cycle(i);
    }
    if (deadlock == 0) {
        printf("no deadlock\n");
    }

}

static void *thread_routine(void *args) {

    while (1) {
        sleep(2);
        check_dead_lock();
    }
}


void start_check(void) {
    tg = (struct task_graph*)malloc(sizeof(struct task_graph));
    tg->num = 0;
    tg->lockidx = 0;

    pthread_t tid;
    pthread_create(&tid, NULL, thread_routine, NULL);
}


#if 1

int search_lock(uint64 lock) {

    int i = 0;
    for (i = 0;i < tg->lockidx;i ++) {
        if (tg->locklist[i].lock_id == lock) {
            return i;
        }
    }
    return -1;
}

int search_empty_lock(uint64 lock) {
    int i = 0;
```

```
    for (i = 0;i < tg->lockidx;i ++) {
        if (tg->locklist[i].lock_id == 0) {
            return i;
        }
    }
    return tg->lockidx;
}

#endif

int inc(int *value, int add) {
    int old;
    __asm__ volatile(
        "lock;xaddl %2, %1;"
        : "=a"(old)
        : "m"(*value), "a" (add)
        : "cc", "memory"
    );
    return old;
}



void print_locklist(void) {
    int i = 0;
    printf("print_locklist: \n");
    printf("----------------------\n");
    for (i = 0;i < tg->lockidx;i ++) {
        printf("threadid : %ld, lockid: %ld\n", tg->locklist[i].id,
tg->locklist[i].lock_id);
    }
    printf("----------------------\n\n\n");
}

void lock_before(uint64 thread_id, uint64 lockaddr) {
    int idx = 0;
    // list<threadid, toThreadid>
    for(idx = 0;idx < tg->lockidx;idx ++) {
        if ((tg->locklist[idx].lock_id == lockaddr)) {
            struct source_type from;
            from.id = thread_id;
            from.type = PROCESS;
            add_vertex(from);
```

```
            struct source_type to;
            to.id = tg->locklist[idx].id;
            tg->locklist[idx].degress++;
            to.type = PROCESS;
            add_vertex(to);

            if (!verify_edge(from, to)) {
                add_edge(from, to); //
            }
        }
    }
}

void lock_after(uint64 thread_id, uint64 lockaddr) {
    int idx = 0;
    if (-1 == (idx = search_lock(lockaddr))) {  // lock list opera

        int eidx = search_empty_lock(lockaddr);

        tg->locklist[eidx].id = thread_id;
        tg->locklist[eidx].lock_id = lockaddr;

        inc(&tg->lockidx, 1);

    }
    else
    {
        struct source_type from;
        from.id = thread_id;
        from.type = PROCESS;

        struct source_type to;
        to.id = tg->locklist[idx].id;
        tg->locklist[idx].degress --;
        to.type = PROCESS;

        if (verify_edge(from, to))
            remove_edge(from, to);
        tg->locklist[idx].id = thread_id;
    }
}
```

```
void unlock_after(uint64 thread_id, uint64 lockaddr) {
    int idx = search_lock(lockaddr);

    if (tg->locklist[idx].degress == 0) {
        tg->locklist[idx].id = 0;
        tg->locklist[idx].lock_id = 0;
        //inc(&tg->lockidx, -1);
    }
}

int pthread_mutex_lock(pthread_mutex_t *mutex) {

    pthread_t selfid = pthread_self(); //

    lock_before(selfid, (uint64)mutex);
    pthread_mutex_lock_f(mutex);
    lock_after(selfid, (uint64)mutex);

}

int pthread_mutex_unlock(pthread_mutex_t *mutex) {

    pthread_t selfid = pthread_self();
    pthread_mutex_unlock_f(mutex);
    unlock_after(selfid, (uint64)mutex);
}

static int init_hook() {
    pthread_mutex_lock_f = dlsym(RTLD_NEXT, "pthread_mutex_lock");
    pthread_mutex_unlock_f = dlsym(RTLD_NEXT,
"pthread_mutex_unlock");
}

#if 1

pthread_mutex_t mutex_1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex_2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex_3 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex_4 = PTHREAD_MUTEX_INITIALIZER;

void *thread_rountine_1(void *args)
```

```
{
    pthread_t selfid = pthread_self(); //

    printf("thread_routine 1 : %ld \n", selfid);

    pthread_mutex_lock(&mutex_1);
    sleep(1);
    pthread_mutex_lock(&mutex_2);

    pthread_mutex_unlock(&mutex_2);
    pthread_mutex_unlock(&mutex_1);

    return (void *)(0);
}

void *thread_rountine_2(void *args)
{
    pthread_t selfid = pthread_self(); //

    printf("thread_routine 2 : %ld \n", selfid);

    pthread_mutex_lock(&mutex_2);
    sleep(1);
    pthread_mutex_lock(&mutex_3);

    pthread_mutex_unlock(&mutex_3);
    pthread_mutex_unlock(&mutex_2);

    return (void *)(0);
}

void *thread_rountine_3(void *args)
{
    pthread_t selfid = pthread_self(); //

    printf("thread_routine 3 : %ld \n", selfid);

    pthread_mutex_lock(&mutex_3);
    sleep(1);
    pthread_mutex_lock(&mutex_4);

    pthread_mutex_unlock(&mutex_4);
```

```
    pthread_mutex_unlock(&mutex_3);

    return (void *)(0);
}

void *thread_rountine_4(void *args)
{
    pthread_t selfid = pthread_self(); //

    printf("thread_routine 4 : %ld \n", selfid);

    pthread_mutex_lock(&mutex_4);
    sleep(1);
    pthread_mutex_lock(&mutex_1);

    pthread_mutex_unlock(&mutex_1);
    pthread_mutex_unlock(&mutex_4);

    return (void *)(0);
}

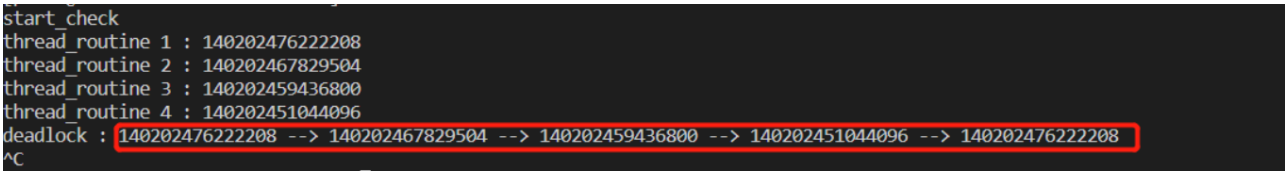

int main()
{
    init_hook();
    start_check();
    printf("start_check\n");
    pthread_t tid1, tid2, tid3, tid4;
    pthread_create(&tid1, NULL, thread_rountine_1, NULL);
    pthread_create(&tid2, NULL, thread_rountine_2, NULL);
    pthread_create(&tid3, NULL, thread_rountine_3, NULL);
    pthread_create(&tid4, NULL, thread_rountine_4, NULL);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    pthread_join(tid3, NULL);
    pthread_join(tid4, NULL);

    return 0;
}

#endif
```

结果如下



能够检测出死锁的产生。

Challenge 2 ： 参考Linux的RCU机制，在ucore中实现简化的RCU机制

在ucore 下实现下Linux的RCU同步互斥机制。可阅读相关Linux内核书籍或查询网上资料，可了解RCU的设计实现细节，然后简化实现在ucore中。 要求有实验报告说明你的设计思路，并提供测试用例。下面是一些参考资料：

- http://www.ibm.com/developerworks/cn/linux/l-rcu/
- http://www.diybl.com/course/6_system/linux/Linuxjs/20081117/151814.html

RCU简介

RCU 的全称是（Read-Copy-Update），意在读写-复制-更新，在 Linux 提供的所有内核互斥的设施当中属于一种免锁机制。在之前讨论过的读写自旋锁（rwlock）、顺序锁（seqlock）一样，RCU 的适用模型也是读写共存的系统。

- 读写自旋锁：读者和写者互斥，读者和读者共存，写者和写者互斥。（偏向读者）
- 顺序锁：写者和写者互斥，写者直接打断读者（偏向写者）

RCU 与他们不同，它的读取和写入操作无需考虑两者之间的互斥问题。

之前的锁分析中，可以知道，加锁、解锁都涉及内存操作，同时伴有内存屏障引入，这些都会导致锁操作的系统开销变大，在此基础之上， 内核在 Kernel 的 2.5 版本引入了 RCU 的免锁互斥访问机制。因此RCU实际上是一种改进的 rwlock，读者几乎没有什么同步开销，它不需要锁，不使用原子指令。不需要锁也使得使用更容易，因为死锁问题就不需要考虑了。写者的同步开销比较大，它需要延迟数据结构的释放，复制被修改的数据结构，它也必须使用某种锁机制同步并行的其它写者的修改操作。读者必须提供一个信号给写者以便写者能够确定数据可以被安全地释放或修改的时机。有一个专门的垃圾收集器来探测读者的信号，一旦所有的读者都已经发送信号告知它们都不在使用被RCU保护的数据结构，垃圾收集器就调用回调函数完成最后的数据释放或修改操作。

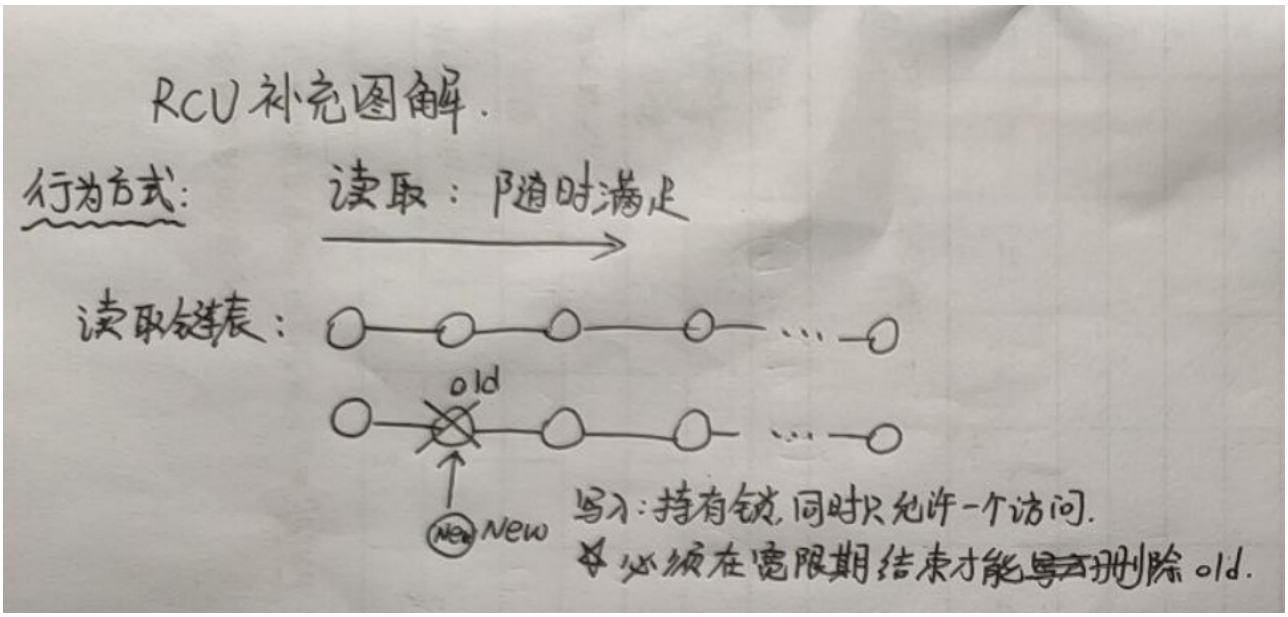
RCU行为方式

1、随时可以拿到读锁，即对临界区的读操作随时都可以得到满足

2、某一时刻只能有一个人拿到写锁，多个写锁需要互斥，写的动作包括 拷贝–修改–宽限窗口到期后删除原值

3、临界区的原始值为m1,如会有人拿到写锁修改了临界区为m2,则在写锁修改临界区之后拿到的读锁获取的临界区的值为m2,之前获取的为m1,这通过原子操作保证

对比发现RCU读操作随时都会得到满足，但写锁之后的写操作所耗费的系统资源就相对比较多了，并且只有在宽限期之后删除原资源。



针对对象

RCU 保护的对象是指针。这一点尤其重要.因为指针赋值是一条单指令.也就是说是一个原子操作.因它更改指针指向没必要考虑它的同步.只需要考虑cache的影响。

内核中所有关于 RCU 的操作都应该使用内核提供的 RCU 的 APIs 函数完成，这些 APIs 主要集中在指针和链表的操作。

实现原理

在RCU的实现过程中，我们主要解决以下问题：

1，在读取过程中，另外一个线程删除了一个节点。删除线程可以把这个节点从链表中移除，但它不能直接销毁这个节点，必须等到所有的读取线程读取完成以后，才进行销毁操作。RCU中把这个过程称为宽限期（Grace period）。

2，在读取过程中，另外一个线程插入了一个新节点，而读线程读到了这个节点，那么需要保证读到的这个节点是完整的。这里涉及到了发布-订阅机制（Publish-Subscribe Mechanism）。

3， 保证读取链表的完整性。新增或者删除一个节点，不至于导致遍历一个链表从中间断开。但是RCU并不保证一定能读到新增的节点或者不读到要被删除的节点。

具体代码

1、宽限期

```
void foo_read(void)
{
    rcu_read_lock();//宽限期开始
    foo *fp = gbl_foo;
    if ( fp != NULL )
        dosomething(fp->a,fp->b,fp->c);//读取线程在完成读取操作
    rcu_read_unlock();//宽限期结束
}

void foo_update( foo* new_fp )
{
    spin_lock(&foo_mutex);//自旋锁
    foo *old_fp = gbl_foo;
    gbl_foo = new_fp;//将旧的节点替换为新的节点
    spin_unlock(&foo_mutex);//自旋锁
    synchronize_rcu();
    //调用该函数意味着一个宽限期的开始，而直到宽限期结束，该函数才会返回
    kfee(old_fp);//宽限期结束后，销毁旧的节点
}
```

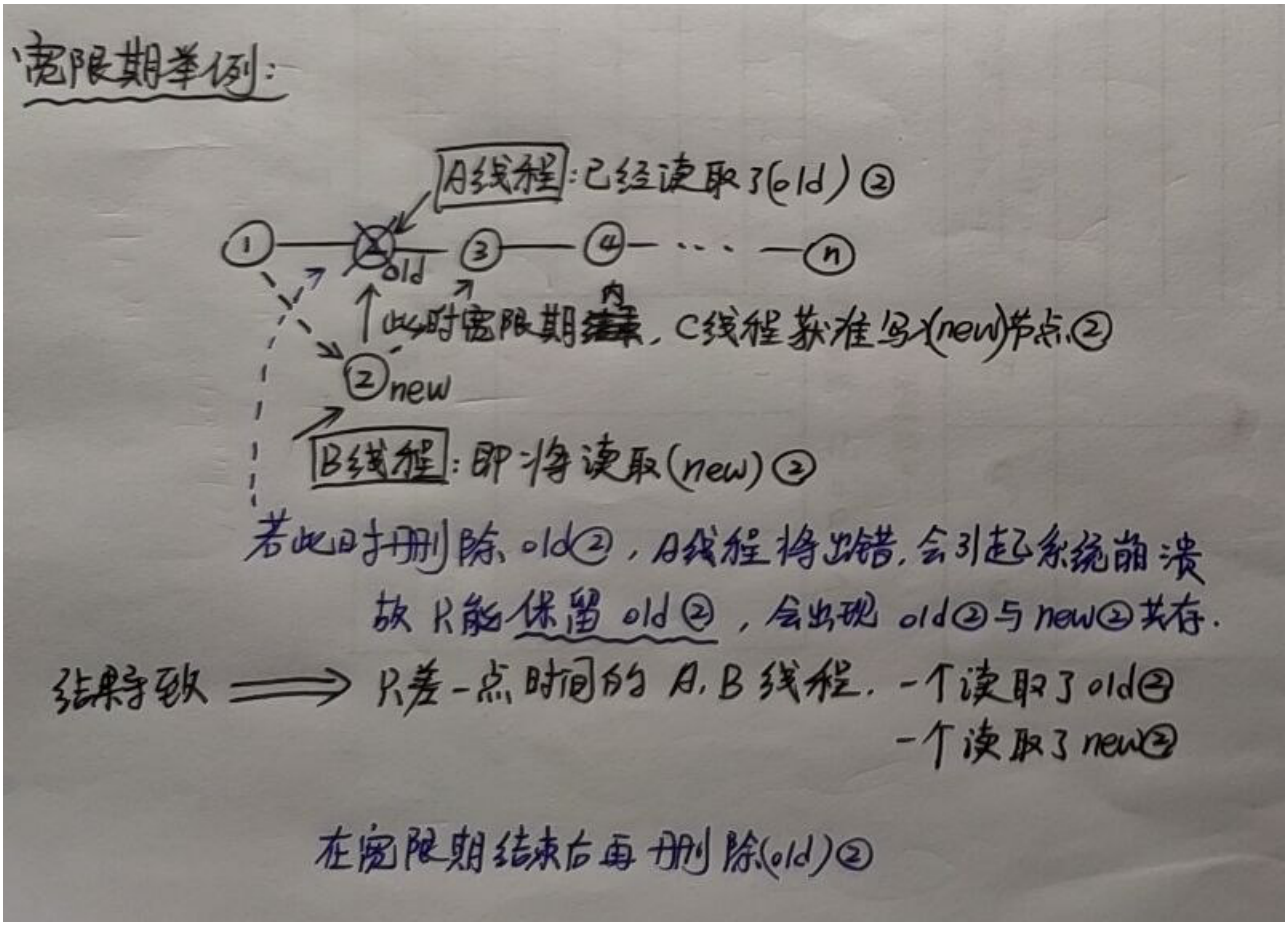
该种机制不保证读取线程在读取时究竟读取到的是被更新过的新节点还是原来的旧节点。我们做到的只是解决读写并发的问题。

试想象如下场景：

在 synchronize_rcu 之前，线程A可能得到了旧的 gbl_foo，也就是 foo_update 中的 old_fp，如果不等它运行结束，就调用 kfee(old_fp)，极有可能造成系统崩溃。

而线程B在synchronize_rcu 之后运行，此时它已经不可能得到 old_fp，此次的kfee将不对它产生影响。

所以宽限期是必要的。虽然旧节点可能没用了，还是得保留，必须在宽限期结束后才能删除旧节点。



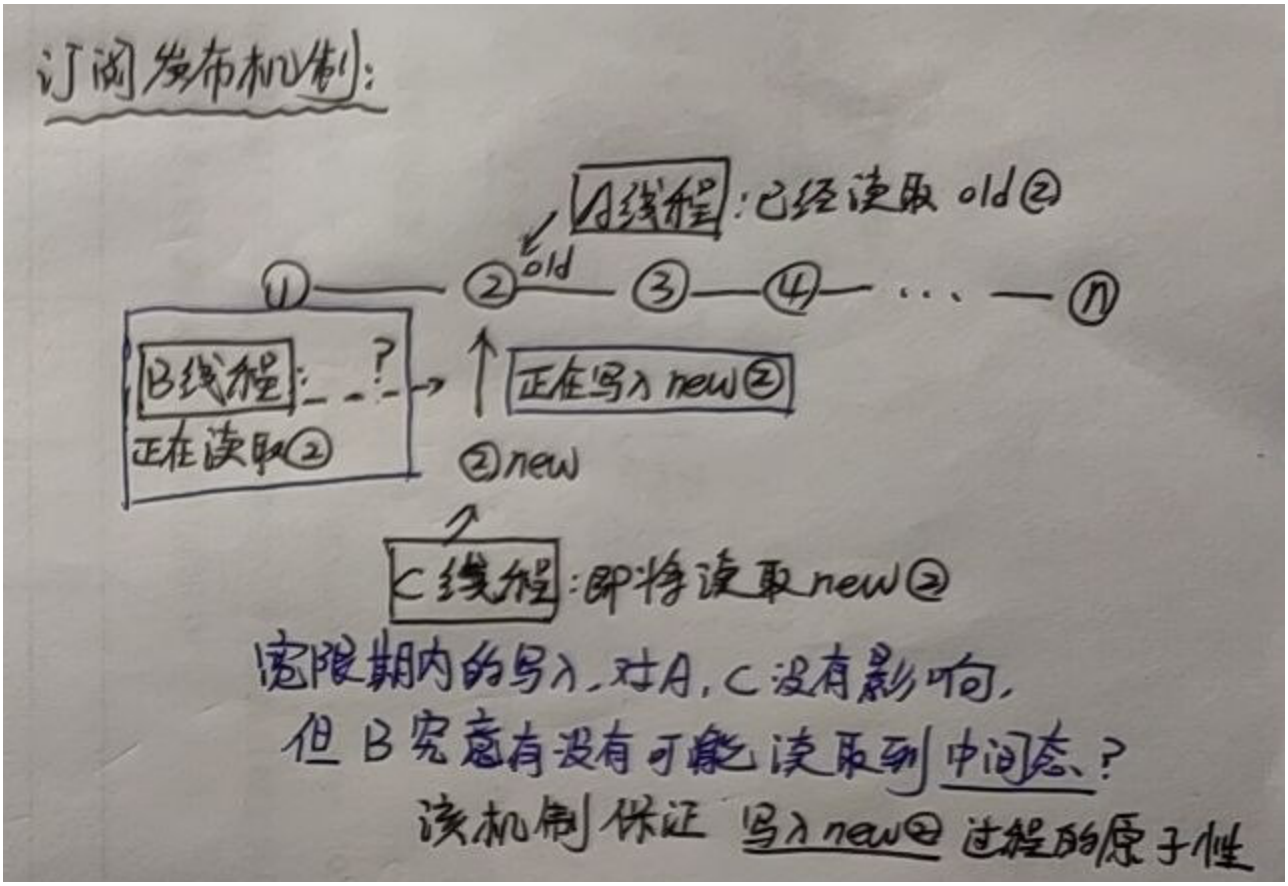
2、订阅--发布机制

★这里涉及到了编译原理的内存屏障，不再深入展开。简要来说就是通过该机制保证插入的节点符合原子性（全或无）。

```
#define rcu_assign_pointer(p, v)
        __rcu_assign_pointer((p), (v), __rcu)

#define __rcu_assign_pointer(p, v, space)
        do {
                smp_wmb();
                (p) = (typeof(*v) __force space *)(v);
        } while (0)
```

我们可以看到它的实现只是在赋值之前加了优化屏障 smp_wmb来确保代码的执行顺序。另外就是宏中用到的__rcu，只是作为编译过程的检测条件来使用的。



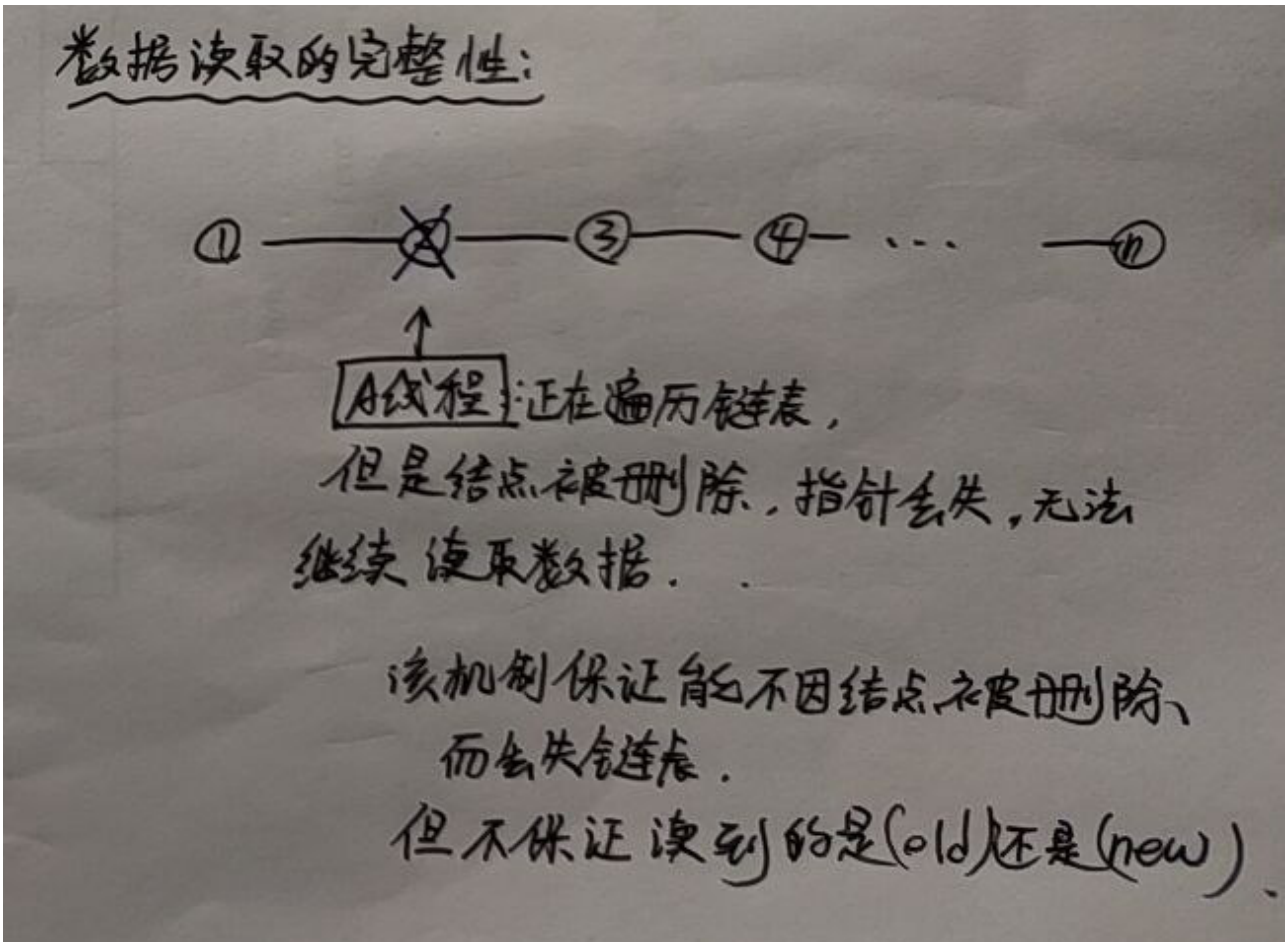
3、数据读取的完整性

★这里涉及到了编译原理的内存屏障，不再深入展开。简要来说就是通过该机制保证访问链表时是完整的，不至于因为节点的删除或新增而断开。

为了解决读取完整性的问题，我们调用一个发布语义的原生接口，rcu_assign_ pointer()：

```
#define rcu_assign_pointer(p, v)
{
    uintptr_t _r_a_p__v = (uintptr_t)(v);
    if (__builtin_constant_p(v) && (_r_a_p__v) == (uintptr_t)NULL)
//在必要时插入一个内存屏障
        {
            WRITE_ONCE((p), (typeof(p))(_r_a_p__v));
        }
    else {          //关闭编译器在赋值时的非顺序编译优化，保证赋值时已经初始化
了。
            smp_store_release(&p,
RCU_INITIALIZER((typeof(p))_r_a_p__v));
            }
}
```

这个接口能够保证从编译器和CPU层面上gp被赋值前，p指向的字段能够赋值完成。



实验总结

lab7的学习使我收获颇丰，但这对于线程同步相关领域的学习还是远远不够。同属于进程间通信IPC领域的经典问题除了哲学家就餐问题外，还有读者/写者问题等；对于线程安全问题，除了使用休眠/唤醒进行线程上下文切换的阻塞的方式之外，还有使用CAS等重试的方法；除了信号量、条件变量等基于单机系统内的线程同步方式外，还有基于分布式系统，通过网络进行多机器线程同步的机制等等。虽然不够了解的知识还有很多，但通过ucore操作系统的学习，为我学习相关的领域知识打下了基础，也给了我相信最终能融会贯通这些知识的信心。

相关知识点

1、原子操作（**Atomic Operator**）

- 原子操作是指一次不存在任何中断或失效的操作
- 该操作只有两种情况
    - 操作成功执行
- 操作没有执行

    > 不存在出现部分执行的情况

- 操作系统需要利用同步机制在并发执行的同时，保存一些操作是原子操作。

2、进程的交互关系

相互感知的程度	交互关系	进程间的影响
相互不感知（完全不了解其他进程的存在）	独立	一个进程的操作对其他进程的结果无影响
间接感知（双方都与第三方交互，例如数据共享）	通过共享进行协作	一个进程的结果依赖于共享资源的状态
直接感知（双方直接交互，例如通信）	通过通信进行协作	一个进程的结果依赖于从其他进程获得的信息

进程之间可能出现三种关系：

- 互斥（mutual exclusion）：一个进程占用资源，其他进程不能使用
- 死锁（deadlock）：多个进程占用部分资源，形成循环等待
- 饥饿（starvation）：其他进程可能轮流占用资源，一个进程一直得不到资源

3、临界区

a. 相关区域的概念

- 临界区（critical section）：进程中访问临界资源的一段需要互斥执行的代码。
- 进入区（entry section）：检查可否进入临界区的一段代码。如果可以进入，则设置“正在访问临界区”标志
- 退出区（exit section）: 清除标志
- 剩余区（remainder section）: 代码中的其余部分

b. 临界区的访问规则

空闲则入、忙则等待、有限等待、让权等待（可选）

> 让权等待：让不能进入临界区的进程暂时释放*CPU*资源。

与参考答案的比较

练习1

不需要编写代码，主要是理解ucore中使用开关中断实现互斥的方式以及信号量的实现，并理解哲学家就餐问题以及基于信号量的解决方案。

练习2

与答案一致。主要是根据注释完成条件变量的实现。由于对条件变量与管程的结合实现并不熟悉，代码部分主要是依靠注释完成，然后通过实验指导和条件变量以及管程的实现代码理解条件变量是怎样实现的。尤其是对管程及管程中配合条件变量的next信号量进行理解。哲学家就餐问题的基于条件变量的解决方法，与信号量解决方法是一致的。

涉及知识点

- 并发问题
- 信号量的实现与使用
- 条件变量的实现与使用
- 哲学家就餐问题的解决
- 管程机制
- 未涉及的知识点

- 虚拟内存
- 进程及进程调度
- 文件系统

参考资料

- https://github.com/AngelKitty/review_the_national_post-graduate_entrance_examination/blob/master/books_and_notes/professional_courses/operating_system/sources/ucore_os_lab/docs/lab_report/lab7/lab7%20%E5%AE%9E%E9%AA%8C%E6%8A%A5%E5%91%8A.md
- RCU部分https://blog.csdn.net/JustinAustin/article/details/123429242
- A橙_学长的指导https://blog.csdn.net/Aaron503/article/details/125112863
- 中山大学参考https://blog.csdn.net/dingdingdodo/article/details/100624931
- 实验总结部分https://www.likecs.com/show-121129.html
- 死锁监测https://blog.csdn.net/weixin_46935110/article/details/127036005

附录：**challenge**总结图

